

# Data Structure

## 05. 큐와 덱

5주차: Queue & Deque

# 목차 INDEX

## 01 큐

- Queue
- Enqueue & Dequeue
- Stack & Queue

## 02 큐의 구현

- 배열 기반 Queue
- Queue의 복잡도
- Circular Queue

## 03 데크

- Deque
- Stack, Queue와의 관계
- 주요 연산과 활용

01



Queue

## 이번 시간에 본 거: Stack

### 4주차: Stack (스택)

LIFO (Last In, First Out)

가장 나중에 들어온 데이터가 가장 먼저 나가는 구조

### 5주차: Queue (큐)

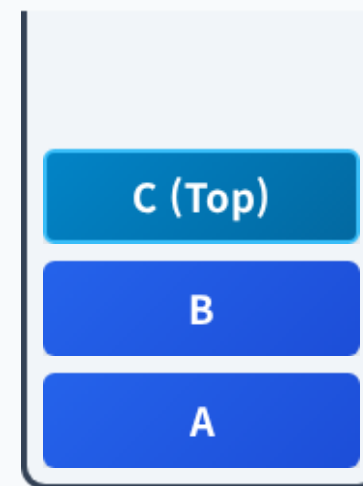
FIFO (First In, First Out)

가장 먼저 들어온 데이터가 가장 먼저 나가는 구조 (이게 큐 설명 끝임)

[출구] ← [A] [B] [C] ← [입구]

A가 먼저 들어왔으므로 A가 먼저 처리됨

TOP ↓



입력 순서: A → B → C

출력 순서: C → B → A

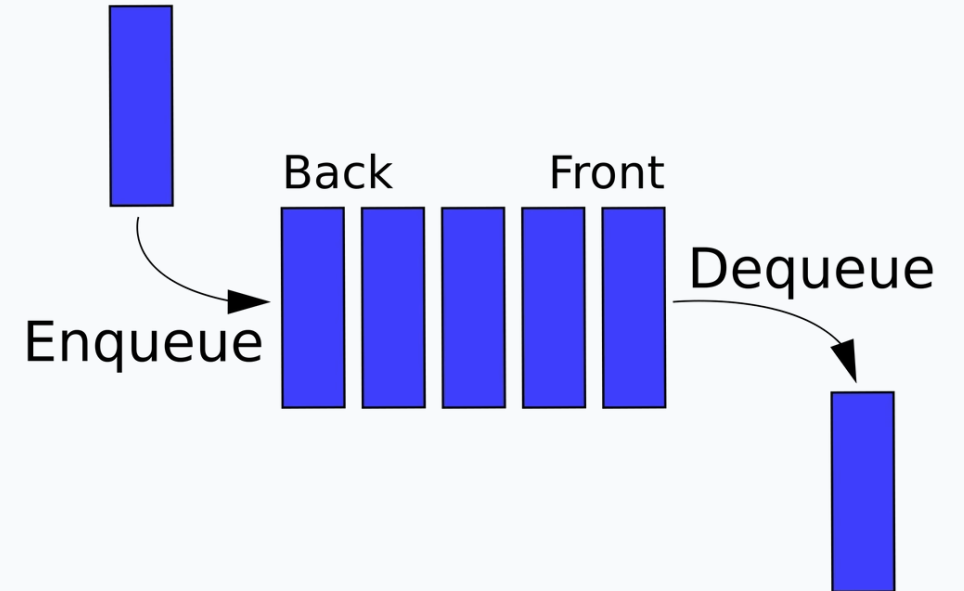
# Enqueue & Dequeue

## Queue (큐)

"먼저 들어온 데이터가 먼저 나오는 FIFO 선형 자료구조"

큐로 들어오는걸 **Enqueue**,  
큐에서 나가는걸 **Dequeue**라 함

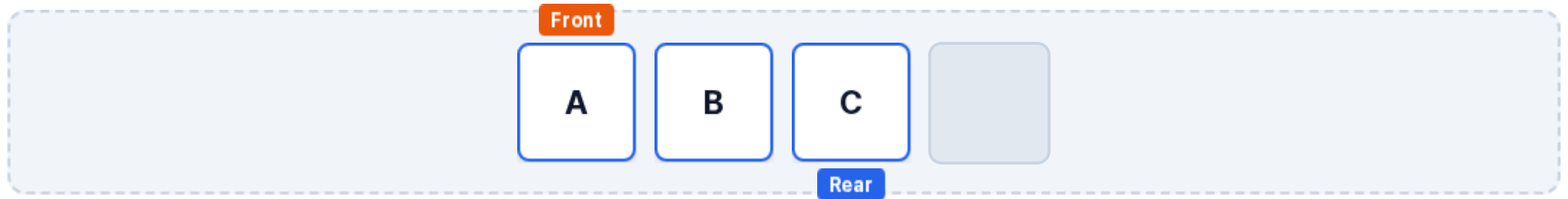
FIFO : First In, First Out (선입선출)



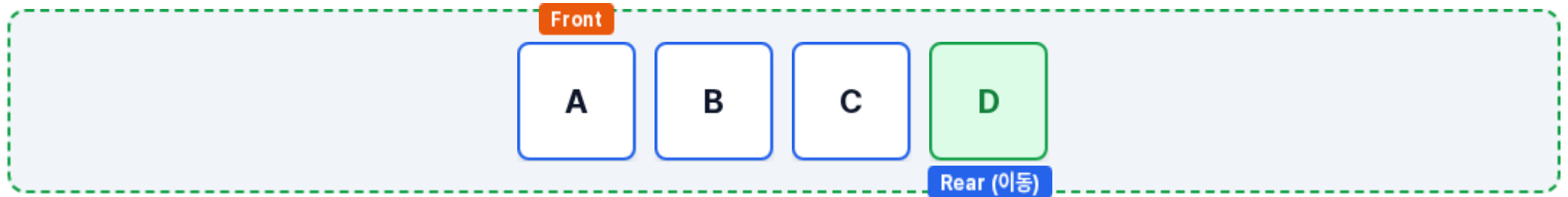
## Enqueue

Enqueue는 데이터가 Queue의 Rear(끝)쪽으로 들어오는 연산

1. 초기 상태:



2. Enqueue(D):



✓Enqueue가 수행되면 Rear 포인터가 오른쪽으로 1칸 이동하면서 새 데이터를 가리킴

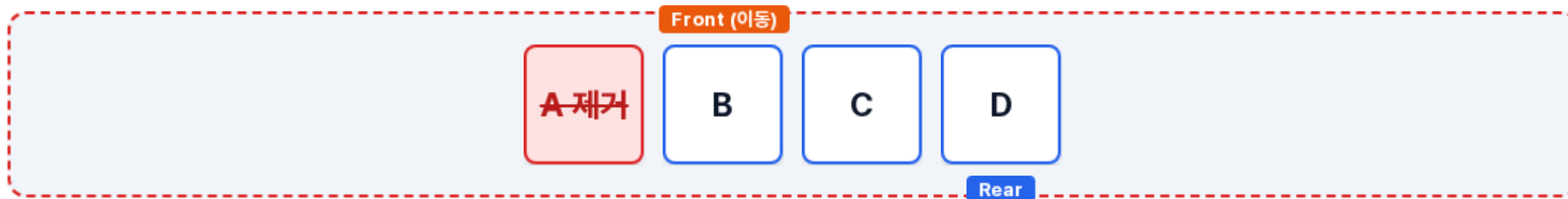
## Deque

Deque는 Queue에 가장 먼저 들어온 Front(시작)쪽 데이터를 제거하는 연산

1. Dequeue 전:



2. Dequeue():



✓Dequeue가 수행되면 A가 제거되면서 Front 포인터가 다음 요소(B) 위치로 1칸 이동함

# Stack & Queue

| 비교 항목     | Stack (스택)               | Queue (큐)                     |
|-----------|--------------------------|-------------------------------|
| 처리 순서     | LIFO (후입선출)              | FIFO (선입선출)                   |
| 데이터 삽입 위치 | Top (맨 위)                | Rear / Back (맨 뒤)             |
| 데이터 삭제 위치 | Top (맨 위)                | Front (맨 앞)                   |
| 핵심 연산     | push(), pop()            | enqueue(), dequeue()          |
| 주요 활용     | 실행 취소(Undo), 뒤로가기, 재귀 호출 | 작업 대기열, 프린터 스푼, 너비 우선 탐색(BFS) |

02

# 큐의 구현

How to Queue

## 02 큐의 구현

### Queue의 이상적인 복잡도

Enqueue

$O(1)$

Rear 위치에 바로 추가

Dequeue

$O(1)$

Front 위치에서 바로 제거

Front / Peek

$O(1)$

Front 위치 데이터 조회

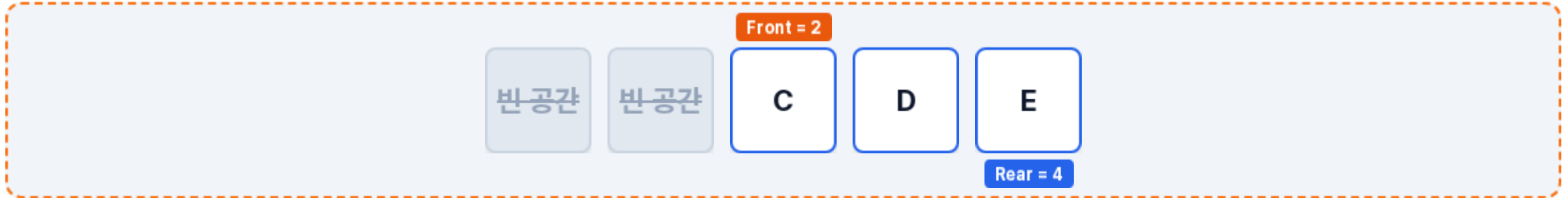
무슨 자료구조로 큐를 구현하냐에 따라 약간씩 달라질 수 있음



크기가 5인 고정 배열로 큐(Linear Queue)를 구현한 모습  
Front랑 Rear 포인터로 시작과 끝을 가리키면 됨

## 02 큐의 구현

### Linear Queue의 문제



Linear Queue는 Rear 가 배열 끝에 도달하면 Queue가 꽉 찼다고 판단함  
0,1번 인덱스가 비어있음에도 불구하고 새로운 데이터 Enqueue 불가



↓ 모든 데이터 이동 (Shift)



✕ 왼쪽처럼 Dequeue 할 때마다  
모든 요소를 앞으로 땡기면

데이터가 N개라면, 1번 Dequeue 할 때마다 N-1개 데이터를 이동해야 함

**시간복잡도:  $O(N)$**

이 방식은  $O(1)$ 이어야 할 Queue의 핵심 연산을 개느리게 만듦

## 02 큐의 구현

# Circular Queue

### 배열의 처음과 끝을 연결하자

선형 배열의 마지막 인덱스 다음을  
다시 인덱스 0으로 연결하는 개념 구조

- 데이터 이동(Shift) 없이 Front와 Rear 포인터만 이동
- 앞쪽의 빈 공간을 재사용 가능
- 모든 Enqueue, Dequeue 연산을  $O(1)$ 에 처리



모듈로(Modulo, %) 연산 활용  
(나머지 연산자임)

$(\text{index} + 1) \% \text{capacity}$

### Front 이동 (Dequeue 시)

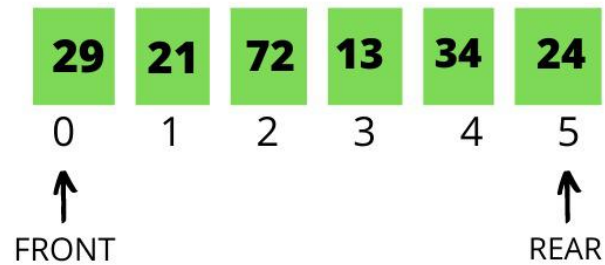
```
front = (front + 1) % capacity
```

### Rear 이동 (Enqueue 시)

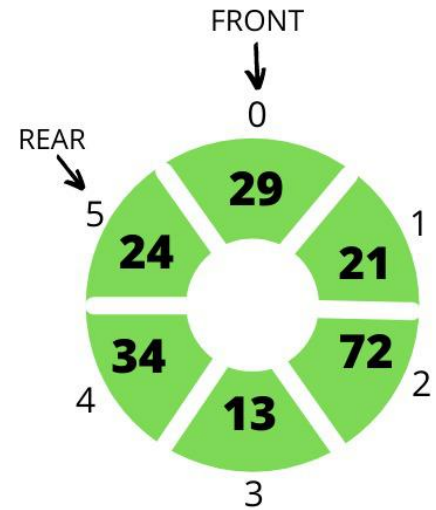
```
rear = (rear + 1) % capacity
```

예시: 배열의 크기가 5이고, Rear가 4일 때  $\rightarrow (4 + 1) \% 5 = 0$  (배열의 맨 처음 0번 인덱스로 회전)

# Circular Queue



**LINEAR QUEUE**



**CIRCULAR QUEUE**

⊘ Empty (비어 있음)

**front == rear**

Queue에 저장된 데이터가 0개인 상태

👉 Full (가득 참 - 1칸 비워두기 전략)

**$(rear + 1) \% capacity == front$**

구분을 위해 배열의 1칸은 항상 사용하지 않고 비워둠

03

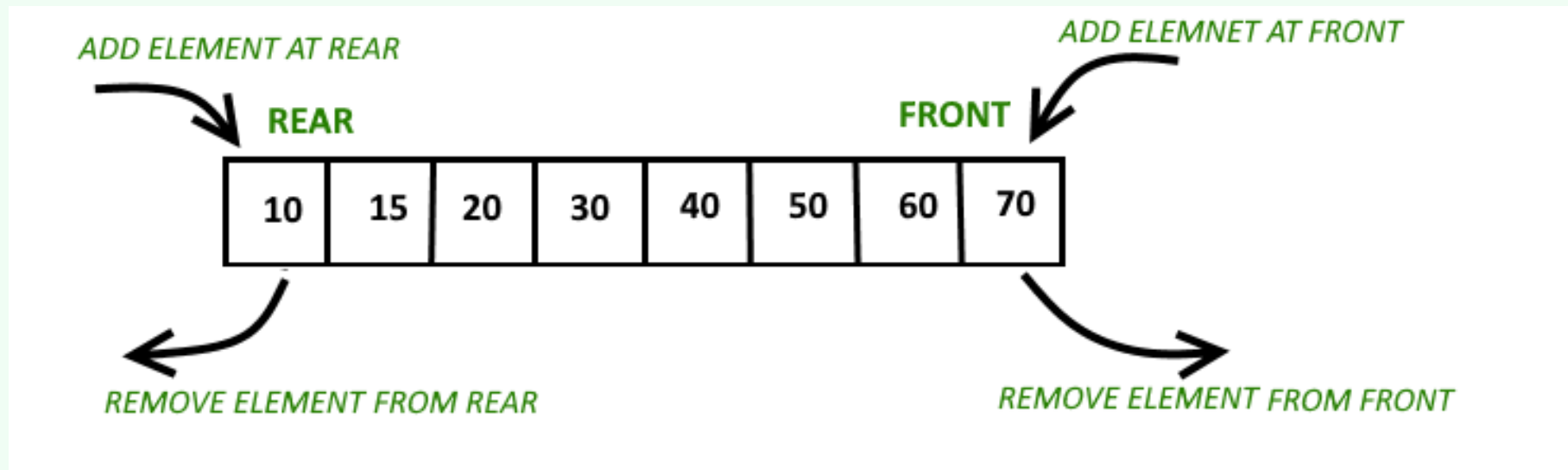
덱

Deque

## Deque

### Deque (Double-Ended Queue)

양쪽 끝(Front & Rear) 모두에서 데이터 삽입과 삭제가 가능한 큐



## Stack & Queue & Deque

### Stack 구현

한쪽 끝만 사용

```
push → push_back() pop →  
pop_back()
```

### Queue 구현

양쪽 끝을 분리하여 사용

```
enqueue → push_back() dequeue →  
pop_front()
```

### Deque 독자 사용

양쪽에서 자유롭게 입출력

```
양쪽 추가/제거 모두 O(1)
```

```
push_front()  
pop_front()
```



A

B

C



```
push_back()  
pop_back()
```

# Deque in collection

- Python 표준 라이브러리 `collections` 모듈에서 제공
- 핵심 특징: 양쪽 끝에서의 추가/삭제 연산이  $O(1)$ 로 매우 빠르게 최적화됨
- 알고리즘 문제 풀이 및 실무에서 Queue를 사용할 때는 `collections`의 `deque` 쓰는게 국룰임

```
from collections import deque # Deque 객체 생성

dq = deque(['A', 'B', 'C'])

# 오른쪽(Rear)에 추가
dq.append('D')

# 왼쪽(Front)에서 제거
item = dq.popleft() # 'A' 반환 (O(1))
```